

「計算機実験 I」 実習 3 (2026-06-10/17)

- 実習 3 の目的
 - 行列・ベクトルを扱うプログラムの作成方法と、数値線形代数ライブラリの利用方法を学ぶ
 - LU 分解を用いた連立一次方程式の求解や行列計算を通して、数値計算アルゴリズムの基礎を理解する
 - 偏微分方程式や非線形方程式の数値解法、および計算性能の評価方法を学ぶ
- グループ発表
 - 6 名ごとのグループに分かれて実施する
 - 【出席】本日 12:10 までに UTOL の「実習 3 グループ発表の記録」に回答すること
- 自習課題: C 言語における行列の扱いと LAPACK を用いた計算
 1. ベクトルや行列を扱うためのユーティリティ関数が (cmatrix.h) に用意されている。サンプルプログラム matrix_example.c や行列・行列積を計算するプログラム multiply.c の中身を見て、その使い方を確認せよ
 2. LU 分解のサンプルプログラム (lu_decomp.c) について、入力データを変更して解がどのように変化するか確認せよ。コンパイルには、cmatrix.h に加えて、dgetrf.h、dgetrs.h が必要である。また、コンパイル時に LAPACK と BLAS をリンク (-llapack -lblas) する必要がある (ハンドブック 2.14.2 節)

```
$ cc lu_decomp.c -o lu_decomp -llapack
$ ./lu_decomp input1.dat
```
- 自習課題: 疑似乱数 (演習時間中に補足説明あり)
 1. random.c は、Mersenne-Twister 乱数発生器 (mersenne_twister.h) により、 $(0, 1)$ の範囲で一様分布する実数乱数を生成するプログラムである。時系列を図示してみよ。種 (seed) を変えて何度か乱数を生成し比較してみよ。また、ヒストグラムも作成し、一様分布になっていることを確認せよ
- レポート課題
 1. lu_decomp.c を参考にして、LU 分解を用いて行列の行列式を計算するプログラムを作成せよ。 $n \times n$ の Vandermonde 行列 ($v_{ij} = x_j^{i-1}$) ($x_1 \cdots x_n$ は実数) の行列式を計算し、厳密な値 $\prod_{1 \leq i < j \leq n} (x_j - x_i)$ と比較せよ。ピボット選択で行を入れ替えると、行列式の符号が反転することに注意せよ。なお、この課題では、LU 分解 (LAPACK の dgetrf 関数、Python の scipy.linalg.lu 関数など) 以外のライブラリ関数は使ってはならない
 2. LU 分解を用いて Dirichlet 型の境界条件のもとでの二次元 Laplace 方程式の解を求めるプログラムを作成せよ。適当な境界条件 [例えば $u(0, y) = \sin(2\pi y)$, $u(1, y) = \sin(\pi y)$, $u(x, 0) = u(x, 1) = 0$] や電荷分布を仮定して解を計算し、Gnuplot の splot コマンド (あるいは Matplotlib の plot_surface など) を用いて解をプロットせよ。また、メッシュ数を増やすと、解の形や計算時間がどのように変化するか調べよ*¹
 3. 20 行 20 列のランダムな実対称行列、複素エルミート行列、実直交行列、複素ユニタリー行列を生成し、さらに、実際に生成された行列がそれらの性質を満たしているかチェックするプログラムを作成・実行せよ。(ヒント: ランダム行列 A から $A + A^t$, $A + A^\dagger$ を作ると対称行列・エルミート行列が得られる) なお、この課題では、QR 分解や特異値分解などのライ

*¹ 計算時間の測り方については、ハンドブック 1.1.7 節参照

ブラリ関数は使ってはならない

4. 1000 行 1000 列のランダム行列 A と長さ 1000 のランダムベクトル $\mathbf{b}$ を生成せよ。連立一次方程式 $A\mathbf{x} = \mathbf{b}$ を LU 分解を用いて解き、解の残差のノルム $|\mathbf{b} - A\mathbf{x}|$ を評価せよ^{*2}。また、LU 分解を使って、1000 個の標準基底 $\mathbf{e}_i$ ($i = 1, \dots, 1000$) について $A\mathbf{x} = \mathbf{e}_i$ を解くことで、 A の逆行列を求め、 $\mathbf{x} = A^{-1}\mathbf{b}$ により解を求めよ。残差のノルムを最初の場合と比較し、どちらが精度が高いか調べ、その理由を考察せよ
5. 非線形連立方程式

$$f_1(x_1, x_2, x_3) = 3x_1 - \cos(x_2x_3) - \frac{1}{2} = 0$$

$$f_2(x_1, x_2, x_3) = x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 = 0$$

$$f_3(x_1, x_2, x_3) = e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0$$

を多次元のニュートン法 (講義 1 スライド p.21) を用いて解け。ヤコビ行列の逆行列をかける代わりに、LU 分解を用いて線形連立方程式を解くこと。LU 分解には、ライブラリ関数を使ってもよい

6. 【発展】 Laplace 方程式の境界値問題を Gauss-Seidel 法、SOR 法で解くプログラムを作成し、計算結果や計算速度を LU 分解・Jacobi 法と比較せよ。また、収束までの回数を Jacobi 法と比較せよ。特に SOR 法の場合、パラメータ ω ($1 < \omega < 2$) の選び方により、どのように収束回数が増えるか観察し、最適な ω の値について考察せよ
 7. 【発展】 行列・行列積の計算を行うサンプルプログラム multiply.c と、BLAS ライブラリの dgemm 関数を使ってそれと等価な計算を行う multiply_dgemm.c の速度を比較せよ。コンパイルには dgemm.h が必要である^{*3}。行列サイズによっては 100 倍以上もの性能差が出ることがある。BLAS で使われている最適化手法について調べ、なぜこのような性能差が出るのか考察せよ
- 6/29 締切のレポート No.2 では、今回のレポート課題から 2 問を選んで回答すること
 - 言語の指定がない課題については、Python や Julia などでもプログラムを作成してもよい。ただし、その場合でも収束の様子などの解析はきちんと行うこと

^{*2} lu_decomp.c を修正して使ってよい

^{*3} ai では、OS 付属の BLAS、LAPACK ではなく、Intel 製の MKL (Math Kernel Library) に含まれる BLAS や LAPACK を利用するのがよい。MKL を使うには、GNU C コンパイラ (cc, gcc) の代わりに Intel C コンパイラ (icx) を使い、-llapack -lblas の代わりに -qmkl を指定してリンクする。例: `icx -O3 multiply_dgemm.c -qmkl`